



NAME : HAMDAN ALI

WEEK 4 : Day 17

DAY 17

Create buttons that change colors and text on click.

Task 17 : Screenshoots Of Output

WEEK 4 - DAY 17 - DOM EVENTS

Signal Panel

Every button here reacts to a click — text, color, or both. Scroll down for exactly how each one is wired.

01 - REACTION BAR

Was this internship writeup helpful?

A reaction bar like the ones on GitHub issues or documentation pages — click to react, click again to undo.

👍 0

💡 0

❤️ 0

🎉 0

😄 0

😞 0

02 - BUTTON LAB

Five ways a click gets handled

Each button below isolates one specific event-handling technique.

Color Cycle

Uses the `.onclick` property — a single handler assigned directly to the element.

Click to cycle color

Stacked Listeners

Two separate `addEventListener` calls on the same button — both run from one click.

Click me twice-over

Fired count: 0

Event Inspector

Reads the actual `event` object passed into the handler.

Click and inspect

No clicks yet.

One-Time Button

Registered with `{ once: true }` — the listener removes itself after firing.

Use me once

Status Toggle

The purest version of the task: one click, new color, new text.

Notifications Off



After Doing Changes:

RESULT:

01 · REACTION BAR

Was this internship writeup helpful?

A reaction bar like the ones on GitHub issues or documentation pages — click to react, click again to undo.

👍 1

👎 1

❤️ 1

🎉 1

😄 1

😞 1

02 · BUTTON LAB

Five ways a click gets handled

Each button below isolates one specific event-handling technique.

Color Cycle

Uses the `.onClick` property — a single handler assigned directly to the element.

Color #2 of 5

Stacked Listeners

Two separate `addEventListener` calls on the same button — both run from one click.

Click me twice-over

Fired count: 1

Event Inspector

Reads the actual `event` object passed into the handler.

Click and inspect

type: "click" · target: <button> · at (1214, 381)

One-Time Button

Registered with `{ once: true }` — the listener removes itself after firing.

Already used

Status Toggle

The purest version of the task: one click, new color, new text.

Notifications Off

EVENT CONSOLE

click → #once-btn ({ once: true }) — listener has now detached itself
click → #inspector-btn (event object) — type: "click" · target: <button> · at (1214, 381)
click → #stacked-btn (listener 2 of 2) — count now 1
click → #stacked-btn (listener 1 of 2) — toggled flash state
click → #onclick-demo-btn (element.onClick property) — set to #2F6FED
click → #confused (addEventListener) — added, count: 1
click → #smile (addEventListener) — added, count: 1

How it's built

Every reaction and every lab button is wired up once, when the page loads — no handler is created or attached inside a loop that runs on every click. The Reaction Bar uses `addEventListener` exclusively, since it needs to support being clicked any number of times. The Button Lab deliberately spreads across different techniques — the `.onClick` property, multiple stacked listeners, the event object, and a self-removing listener — so each one is demonstrated on its own rather than blended together.

Hamdan Ali · Week 4, Day 17 — Full Stack Web Development Internship



EXPLANATION

DAY 17 — DOM EVENTS

Building the Signal Panel

Hamdan Ali · Week 4 · Full Stack Web Development Internship

What I Was Going For

The task was to create buttons that change color and text on click — which is usually one button, one handler, done in five minutes. I wanted to actually dig into what the topic was really about, which is onclick and addEventListener specifically, and the fact that most people learn to use addEventListener without ever seeing why it's the better default. So I split this into two pieces: a reaction bar that's an actual usable component — the kind of thing you'd see on a GitHub issue or under a blog post — and a small lab section where five buttons each isolate one specific way a click gets handled.

Underneath both, there's a live console that prints a real line of text every time any button fires, saying exactly which technique just ran. It felt more honest than writing comments that describe what the code does off to the side — this way you can watch it happen.

The Reaction Bar

Six reaction buttons, each with its own emoji, its own count, and its own glow color when active. Clicking toggles it on, clicking again undoes it — the same pattern GitHub uses on issues and pull requests. All six share one handler, attached once when the page loads, rather than six separate copies of nearly the same code:

```
reactionButtons.forEach((button) => {  
  const reactionName = button.dataset.reaction;  
  const glowColor = button.dataset.glow;
```

```
button.addEventListener('click', () => {  
  const isActive = button.classList.toggle('is-active');  
  reactionCounts[reactionName] += isActive ? 1 : -1;  
  button.style.setProperty('--glow', glowColor);  
});  
});
```

Each button carries its own color in a data-glow attribute in the HTML, so the one shared handler can still give the heart button a pink glow and the thumbs-up button a green one, without an if/else chain checking which button was clicked. Adding a seventh reaction later is just one more button in the markup — the JavaScript doesn't need to know it exists.

Button Lab, Part 1 — The .onclick Property

This one exists specifically to show the property form of onclick, separate from addEventListener — assigning a function straight to element.onclick, which is the JavaScript-side version of the old inline onclick="" attribute, minus mixing markup and behavior together:

```
onclickDemoBtn.onclick = function () {  
  cycleIndex = (cycleIndex + 1) % cycleColors.length;  
  this.style.backgroundColor = cycleColors[cycleIndex];  
  this.textContent = `Color #${cycleColors[cycleIndex]} of ${cycleColors.length}`;  
};
```

The catch with .onclick, and the reason it's worth showing on its own rather than mixing it in elsewhere, is that it can only ever hold one function. Assign a second one and it silently replaces the first — no error, no warning, it just quietly stops doing whatever the first function did. That's the exact limitation the next button demonstrates by not having it.

Button Lab, Part 2 — Stacking Two Listeners

One button, two completely independent addEventListener calls. Both run, in the order they were attached, from a single click — which is the thing .onclick genuinely cannot do:

```
stackedBtn.addEventListener('click', () => {
```

```
stackedBtn.classList.toggle('lab-btn--flash');
});

stackedBtn.addEventListener('click', () => {
  stackedClicks++;
  stackedCountEl.textContent = stackedClicks;
});
```

Neither of these two listeners knows the other exists. One flips a visual state, the other keeps a running count, and they don't interfere with each other at all — which is really the whole point of `addEventListener` existing as a separate mechanism from a single property assignment.

Button Lab, Part 3 — Reading the Event Object

Every function handed to `addEventListener` automatically receives the event itself as its first argument, whether you use it or not. This button actually reads it:

```
inspectorBtn.addEventListener('click', (event) => {
  const details = `type: "${event.type}" · target: <${event.target.tagName.toLowerCase()}> ·
  at (${event.clientX}, ${event.clientY})`;
  inspectorOutput.textContent = details;
});
```

It's a small thing, but seeing `event.type` print out "click", or `event.target` actually identify which element was pressed, makes that object feel like a real, inspectable thing instead of an invisible parameter that's just always there in tutorials without explanation.

Button Lab, Part 4 — A Listener That Removes Itself

`addEventListener` takes an optional third argument, an options object, and `{ once: true }` tells it to automatically detach the handler right after it fires for the first time — no manual `removeEventListener` call needed:

```
onceBtn.addEventListener(
  'click',
```

```
() => {  
  onceBtn.textContent = 'Already used';  
  onceBtn.disabled = true;  
},  
{ once: true }  
);
```

I tested this one specifically to make sure the listener really was gone and not just visually disabled — clicking it a second time (even with the disabled attribute removed for testing) confirmed nothing else fires. That's a meaningfully different guarantee than just checking a flag inside the handler every time.

Button Lab, Part 5 — The Plain Version

The last button is the task exactly as written, with nothing extra layered on: one click, a new background color, new label text.

```
toggleBtn.addEventListener('click', () => {  
  notificationsOn = !notificationsOn;  
  toggleBtn.textContent = notificationsOn ? 'Notifications On' : 'Notifications Off';  
  toggleBtn.classList.toggle('lab-btn--off', !notificationsOn);  
  toggleBtn.classList.toggle('lab-btn--on', notificationsOn);  
});
```

I left this one deliberately simple. After four buttons showing more specific techniques, it felt right to end with the plainest possible version of what the task actually asked for.

A Few Notes on What I Took From This

- `addEventListener` being able to stack multiple handlers on one element, while `.onclick` can only ever hold one, is the actual practical reason it's the safer default — not just a stylistic preference.
- The event object is worth actually reading sometimes instead of ignoring it — `event.target` in particular is useful the moment you're handling clicks on a group of similar elements rather than one specific button.



- `{ once: true }` is a small addition, but it replaces a whole pattern of manually calling `removeEventListener` inside the handler that used it — one less thing to get wrong.
- Writing five buttons that each isolate one idea made the differences between them far clearer than writing one button that tried to demonstrate everything at once would have.

References Used

I used the W3Schools page on HTML DOM events for the exact behavior of `onclick` and `addEventListener`, and watched the linked video to see event handling used in a live, working example.

Article: [w3schools.com/js/js_htmldom_events.asp](https://www.w3schools.com/js/js_htmldom_events.asp)

Video: youtube.com/watch?v=CO_DAXswOrc

Files Submitted

- `index.html` — the Reaction Bar, the five Button Lab cards, and the event console.
- `style.css` — the Signal Panel styling: glowing reaction buttons, lab cards, and the terminal-style console.
- `script.js` — every event-handling technique described above, plus the shared logging function they all write to.

THANK YOU!